

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- greedy algorithms
- divide and conquer

CS 401/MCS 401 Lecture 10
Computer Algorithms I
Jan Verschelde, 10 July 2026

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- greedy algorithms
- divide and conquer

midterm exam policies

- The Midterm Exam happens on Monday 13 July, from 10AM till 11:40AM.
- The exam covers the first 9 lectures.
This corresponds to the first five chapters in the textbook.
Sections not explicitly covered in class will not be on the exam.
- There will not be a makeup Midterm Exam.
- You may skip the Midterm Exam.
The 100 points transfer automatically to the Final exam.

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- **overview of problems, algorithms, data structures**
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- greedy algorithms
- divide and conquer

overview of problems

- 1 stable marriage (L-1, L-3)
- 2 graph traversals (L-3, L-4)
- 3 connected components in graph (L-4, L-5, L-8)
- 4 testing bipartiteness (L-4)
- 5 ordering nodes in a directed acyclic graph (L-5)
- 6 scheduling requests on one or multiple resources (L-5, L-6)
- 7 optimal caching (L-6)
- 8 shortest paths in a graph (L-6)
- 9 spanning trees (L-7)
- 10 counting inversions (L-8)
- 11 closest pair of points (L-8)
- 12 convolutions (L-8, L-9)
- 13 matrix multiplication (L-9)

overview of algorithms

- 1 Gale-Shapley algorithm (L-1, L-3)
- 2 breadth first search (L-3, L-4)
- 3 depth first search (L-3, L-4)
- 4 topological ordering (L-5)
- 5 interval scheduling (L-5)
- 6 interval partitioning (L-5)
- 7 earliest deadline first (L-6)
- 8 farthest in the future (L-6)
- 9 Dijkstra's algorithm (L-6)
- 10 Kruskal's algorithm (L-7)
- 11 Prim's algorithm (L-7)
- 12 reverse delete (L-7)
- 13 merge sort to count inversions (L-8)
- 14 closest pair of points (L-8)
- 15 Karatsuba's integer multiplication (L-8)
- 16 fast convolution (FFT) (L-9)
- 17 Strassen's method (L-9)

overview of data structures

- 1 array and list (L-3)
- 2 queue and stack (L-4)
- 3 adjacency matrix representation of a graph (L-4)
- 4 edge list representation of a graph (L-4, L-5)
- 5 priority queue or heap (L-7)
- 6 union-find data structure (L-7)

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- **overview of discrete mathematics**

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- greedy algorithms
- divide and conquer

overview of discrete mathematics

- 1 asymptotic bounds: big O , Ω , and Θ (L-2)
- 2 logarithmic, sublinear, linear, subquadratic, quadratic, cubic, polynomial, exponential growth (L-2)
- 3 transitivity and sum properties (L-2)
- 4 solving a recurrence by substitution (L-8)
- 5 solving a recurrence by unrolling (L-8)
- 6 the master method to solve recurrences (L-9)

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- greedy algorithms
- divide and conquer

1. derive and prove a cost function

Consider the following problem:

Input: $U = [u_{i,j}]$, an n -by- n matrix, $u_{i,i} \neq 0$,

$\mathbf{b} = (b_1, b_2, \dots, b_n)$, a vector of length n .

Output: $\mathbf{x} = (x_1, x_2, \dots, x_n)$, a vector of length n , with

$$x_i = \frac{1}{u_{i,i}} \left(b_i - \sum_{j=i+1}^n u_{i,j} x_j \right), \quad i = n, n-1, \dots, 1.$$

- 1 Express the cost to compute $\mathbf{x}$ as a function of n .
- 2 Justify your cost function.

derive and prove a cost function

$$x_i = \frac{1}{u_{i,i}} \left(b_i - \sum_{j=i+1}^n u_{i,j} x_j \right), \quad i = n, n-1, \dots, 1.$$

① Express the cost to compute $\mathbf{x}$ as a function of n .

Answer:

$$\begin{aligned} i = n & : 1 \text{ div} \\ i = n - 1 & : 1 \text{ div}, 1 \text{ mul}, 1 \text{ sub} \\ i = n - 2 & : 1 \text{ div}, 2 \text{ mul}, 1 \text{ add}, 1 \text{ sub} \\ i = n - 3 & : 1 \text{ div}, 3 \text{ mul}, 2 \text{ add}, 1 \text{ sub} \\ i = 1 & : 1 \text{ div}, n - 1 \text{ mul}, n - 2 \text{ add}, 1 \text{ sub} \end{aligned}$$

The cost in function of n is

$$n \text{ div} + \sum_{k=0}^{n-1} k \text{ mul} + \sum_{k=0}^{n-2} k \text{ add} + (n - 1) \text{ sub}.$$

derive and prove a cost function

The cost in function of n is

$$n \text{ div} + \sum_{k=0}^{n-1} k \text{ mul} + \sum_{k=0}^{n-2} k \text{ add} + (n-1) \text{ sub}$$

which is equivalent to

$$n \text{ div} + \frac{1}{2}n(n-1) \text{ mul} + \frac{1}{2}(n-1)(n-2) \text{ add} + (n-1) \text{ sub}.$$

2 Justify your cost function.

Answer: In the derivation: $\sum_{k=1}^n k = \frac{1}{2}n(n+1)$ was used.

An alternative answer writes the mathematical notation as a double for loop.

2. compare in the O , Ω , Θ order

Let $f(n) = n\sqrt{100n}$ and $g(n) = n^2$.

- 1 Consider the statements:

$f(n)$ is $O(g(n))$, $f(n)$ is $\Omega(g(n))$, $f(n)$ is $\Theta(g(n))$.

Which statement is true?

For each statement, justify your answer.

- 2 Consider the statements:

$g(n)$ is $O(f(n))$, $g(n)$ is $\Omega(f(n))$, $g(n)$ is $\Theta(f(n))$.

Which statement is true?

For each statement, justify your answer.

compare in the O , Ω , Θ order

Let $f(n) = n\sqrt{100n}$ and $g(n) = n^2$.

❶ Consider the statements:

$f(n)$ is $O(g(n))$, $f(n)$ is $\Omega(g(n))$, $f(n)$ is $\Theta(g(n))$.

Which statement is true?

For each statement, justify your answer.

Answer: $f(n) = 10n^{3/2}$

❶ $f(n)$ is $O(g(n))$: true, as n^2 grows faster than $10n^{3/2}$.

❷ $f(n)$ is $\Omega(g(n))$: false, as n^2 grows faster than $10n^{3/2}$.

❸ $f(n)$ is $\Theta(g(n))$: false, as $f(n)$ is $\Omega(g(n))$ is false.

compare in the O , Ω , Θ order

Let $f(n) = n\sqrt{100n}$ and $g(n) = n^2$.

② Consider the statements:

$g(n)$ is $O(f(n))$, $g(n)$ is $\Omega(f(n))$, $g(n)$ is $\Theta(f(n))$.

Which statement is true?

For each statement, justify your answer.

Answer: $f(n) = 10n^{3/2}$.

① $g(n)$ is $O(f(n))$: false, as $10n^{3/2}$ grows slower than n^2 .

② $g(n)$ is $\Omega(f(n))$: true, as $10n^{3/2}$ grows slower than n^2 .

③ $g(n)$ is $\Theta(f(n))$: false, as $g(n)$ is $O(f(n))$ is false.

3. solving a recurrence

Apply the master method to solve the following recurrences:

- 1 $T(n) = 9T(n/3) + n$
- 2 $T(n) = T(n/2) + 1$
- 3 $T(n) = 2T(n/2) + n \log(n)$

solving a recurrence with the master method

Solve

❶ $T(n) = 9T(n/3) + n.$

Answer: We have the following:

❶ $a = 9, b = 3, f(n) = n$

❷ $n^{\log_b(a)} = n^{\log_3(9)} = n^2$

❸ $\epsilon = 1, \text{ as } f(n) = n^{2-1} = n$

Case (1) applies: $T(n)$ is $\Theta(n^2)$.

solving a recurrence with the master method

Solve

$$② \quad T(n) = T(n/2) + 1.$$

Answer: We have the following:

$$① \quad a = 1, b = 2, f(n) = 1$$

$$② \quad n^{\log_b(a)} = n^{\log_2(1)} = n^0 = 1$$

Case (2) applies: $T(n)$ is $\Theta(\log(n))$.

solving a recurrence with the master method

Solve

③ $T(n) = 2T(n/2) + n \log(n).$

Answer: We have the following:

① $a = 2, b = 2, f(n) = n \log(n)$

② $n^{\log_2(2)} = n^1$

③ for some $\epsilon > 0$: $f(n)$ is $\Omega(n^{1+\epsilon})$,
and $2n/2 \log(n/2) \leq cn \log(n)$ for some constant c .

Case (3) applies: $T(n)$ is $\Theta(n \log(n))$.

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- **the Gale-Shapley algorithm**
- algorithms on graphs
- greedy algorithms
- divide and conquer

4. the Gale-Shapley algorithm

Consider 4 employers $\{e_1, e_2, e_3, e_4\}$ and 4 applicants $\{a_1, a_2, a_3, a_4\}$.

preferences for employers

e_1 : 1 3 2 4

e_2 : 2 1 4 3

e_3 : 1 3 2 4

e_4 : 4 1 3 2

preferences for applicants

a_1 : 2 3 4 1

a_2 : 1 4 2 3

a_3 : 1 3 4 2

a_4 : 2 1 3 4

Preferences are listed from high to low for employers and applicants.

- 1 Run the Gale-Shapley algorithm on the above input.
Show all stages of the algorithm.
- 2 To demonstrate that the algorithm is efficient, a data structure is needed to determine the next applicant an employers offers to.
What is this data structure?
Illustrate the use of this data structure on the above input.

running the Gale-Shapley algorithm

preferences for employers

e_1 : 1 3 2 4

e_2 : 2 1 4 3

e_3 : 1 3 2 4

e_4 : 4 1 3 2

preferences for applicants

a_1 : 2 3 4 1

a_2 : 1 4 2 3

a_3 : 1 3 4 2

a_4 : 2 1 3 4

- 1 Run the Gale-Shapley algorithm on the above input.
Show all stages of the algorithm.

Answer: The evolution of the set S of committed pairs is below.

- 1 e_1 offers to a_1 , $S = \{(e_1, a_1)\}$
- 2 e_2 offers to a_2 , $S = \{(e_1, a_1), (e_2, a_2)\}$
- 3 e_3 offers to a_1 , a_1 prefers e_3 over e_1 , $S = \{(e_2, a_2), (e_3, a_1)\}$
- 4 e_1 offers to a_3 , $S = \{(e_2, a_2), (e_3, a_1), (e_1, a_3)\}$
- 5 e_4 offers to a_4 , $S = \{(e_2, a_2), (e_3, a_1), (e_1, a_3), (e_4, a_4)\}$

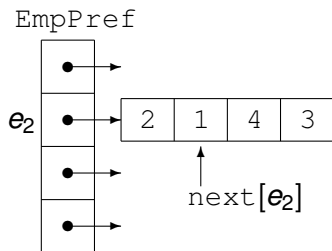
the Gale-Shapley algorithm is efficient

- ② To demonstrate that the algorithm is efficient, a data structure is needed to determine the next applicant an employers offers to.

What is this data structure?

Illustrate the use of this data structure on the above input.

Answer: $\text{next}[e]$ contains the index of next applicant employer e will offer to. The size of the array next is the number of employers. Accessing and updating next happens in time $O(1)$.



- Consider the preferences of e_2 .
- Assume e_2 offered to 2.
- Next time e_2 is free, e_2 offers to 1.
- next is an array which stores for each e the next highest ranked: e offers to $\text{EmpPref}[e, \text{next}[e]]$
- Initially, $\text{next}[e] = 1$ for all e .

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- **algorithms on graphs**
- greedy algorithms
- divide and conquer

5. topological ordering of a directed acyclic graph

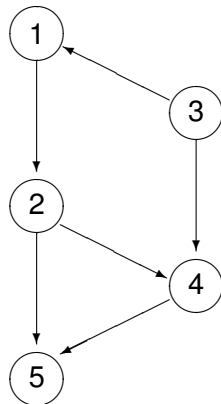
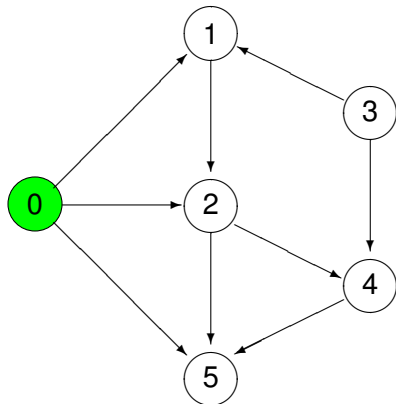
Consider the topological ordering in a DAG.

- 1 Let G be a graph with 6 vertices 0, 1, 2, 3, 4, 5 and edges $E = \{ (0, 1), (0, 2), (0, 5), (1, 2), (3, 1), (2, 4), (2, 5), (3, 4), (4, 5) \}$. Illustrate the algorithm to order the vertices topologically on the above example.
Draw all steps in the execution of the algorithm.
- 2 Is the solution of a topological ordering unique?
Justify your answer.

finding and deleting the source 0

$V = \{0, 1, 2, 3, 4, 5\}$.

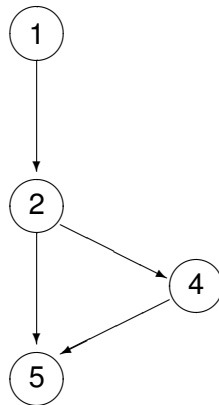
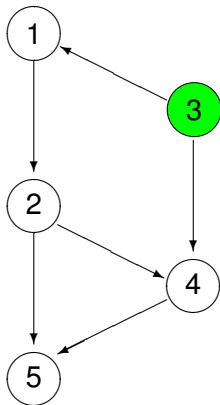
$E = \{ (0, 1), (0, 2), (0, 5), (1, 2), (3, 1), (2, 4), (2, 5), (3, 4), (4, 5) \}$.



finding and deleting the source 3

$V = \{0, 1, 2, 3, 4, 5\}$.

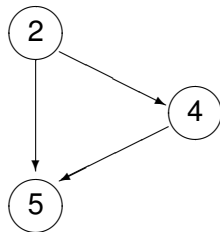
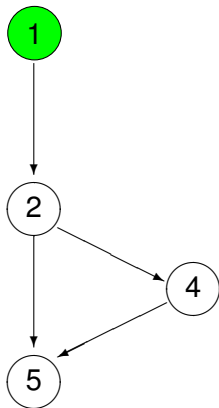
$E = \{ (0, 1), (0, 2), (0, 5), (1, 2), (3, 1), (2, 4), (2, 5), (3, 4), (4, 5) \}$.



finding and deleting the source 1

$V = \{0, 1, 2, 3, 4, 5\}$.

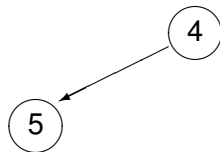
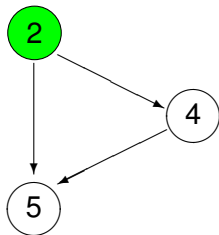
$E = \{ (0, 1), (0, 2), (0, 5), (1, 2), (3, 1), (2, 4), (2, 5), (3, 4), (4, 5) \}$.



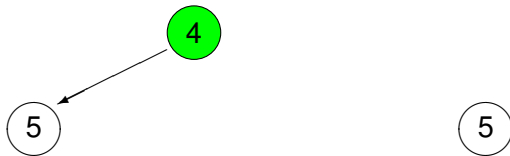
finding and deleting the source 2

$V = \{0, 1, 2, 3, 4, 5\}$.

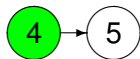
$E = \{ (0, 1), (0, 2), (0, 5), (1, 2), (3, 1), (2, 4), (2, 5), (3, 4), (4, 5) \}$.



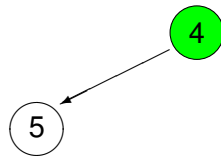
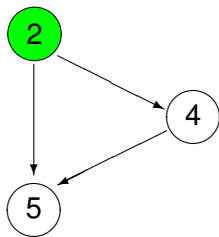
appending the topological order after 4



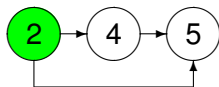
append 5 after 4:



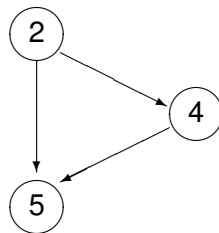
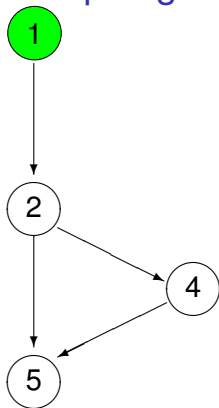
appending the topological order after 2



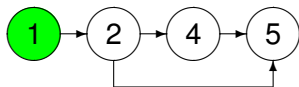
append the topological order after 2:



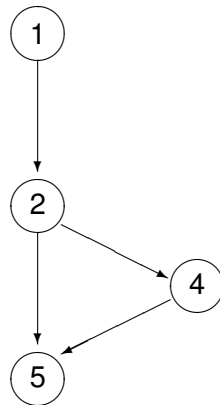
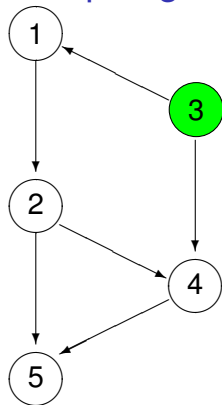
appending the topological order after 1



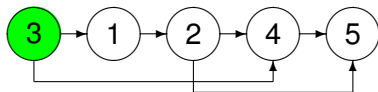
append the topological order after 1:



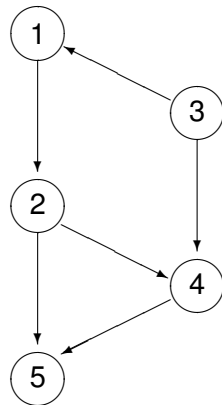
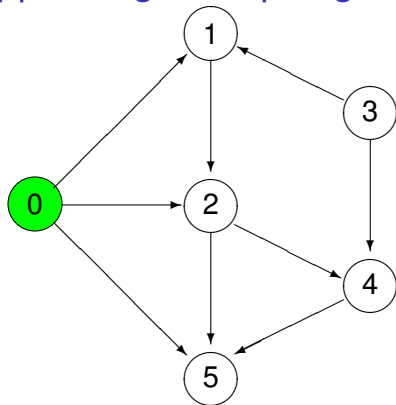
appending the topological order after 3



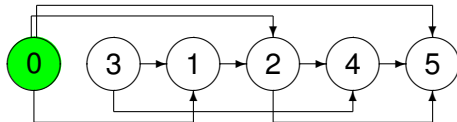
append the topological order after 3:



appending the topological order after 0



append the topological order after 0:



uniqueness of the topological ordering

- 1 Is the solution of a topological ordering unique?
Justify your answer.

Answer:

- 1 There may be more than one source.
In the example, we could have started with 3 instead of 0.
So, the topological ordering is not unique.
- 2 In case of multiple sources, following the order of vertices, the outcome of the topological ordering algorithm is unique.

6. relabeling nodes in a tree

In a tree, assume that the n nodes are labeled

- starting with label 0 at the root, and
- ending at label $n - 1$ of the last visited node in an inorder traversal of the nodes.

Answer the following questions:

- 1 Given any positive label $k < n$, is it possible to reorganize the nodes of the tree so that the node with label k becomes the root?
- 2 Define an algorithm and the data structure used to store the tree.
- 3 Use big O to express the cost of your algorithm.

6. relabeling nodes in a tree

- 1 Given any positive label $k < n$, is it possible to reorganize the nodes of the tree so that the node with label k becomes the root?

Answer:

Yes, proceed as follows, observing the tree may not be binary.

- A tree is a connected graph, where internal nodes have a unique parent and at least one child node.
- For $k > 0$, denote the parent of node k by $p(k)$.
- In the relabeled tree where k is the root, the edge between $p(k)$ and k is now interpreted as the edge between k and its additional child $p(k)$ and k is no longer the child of $p(k)$.

6. relabeling nodes in a tree

- 2 Define an algorithm and the data structure used to store the tree.

Answer:

- A tree with n nodes has $n - 1$ edges.
An adjacency matrix occupies n^2 spots.
An edge list stores every node twice, of size $2n$.
Therefore, we use an edge list representation.
- In the edge list representation, the nodes connected to the root node are in the first list of nodes.
In the array of lists, we swap the edge list k with the edge list at index 0.
- We relabel the nodes in the edge lists,
which requires to visit all edges and all nodes.

6. relabeling nodes in a tree

- ③ Use big O to express the cost of your algorithm.

Answer:

- ① The swapping of the lists in the array happens in constant time.
- ② Visiting the nodes in the edges lists requires $2n$ steps, as every node is stored twice in the edge lists.

The total cost is $O(n)$.

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- greedy algorithms
- divide and conquer

CS 401/MCS 401 Lecture 10
Computer Algorithms I
Jan Verschelde, 10 July 2026

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- **greedy algorithms**
- divide and conquer

7. shortest paths in a graph

Consider the shortest paths in a graph.

- 1 Let G be a weighted graph with 6 vertices 0, 1, 2, 3, 4, 5 with lengths ℓ on edges (i, j) written as $e_{(i,j)} = \ell$ below:
 $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$, $e_{(1,3)} = 15$,
 $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.

Compute the shortest paths originating at 0.

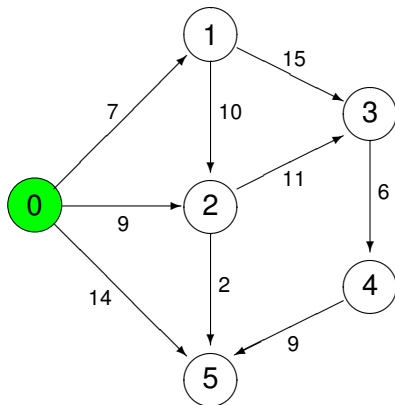
Draw all steps in the execution of the algorithm.

- 2 Use the step wise execution of the algorithm above to illustrate the main argument in the correctness proof of the algorithm.

In particular, if the set of explored vertices equals three, justify why the path to the fourth vertex in the path from 0 will be the shortest path from 0 to the fourth explored vertex.

step 0 of Dijkstra's algorithm

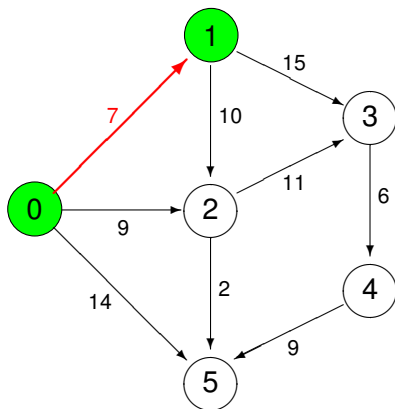
$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$,
 $e_{(1,3)} = 15$, $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.



$$d(0) = 0$$

step 1 of Dijkstra's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$,
 $e_{(1,3)} = 15$, $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.

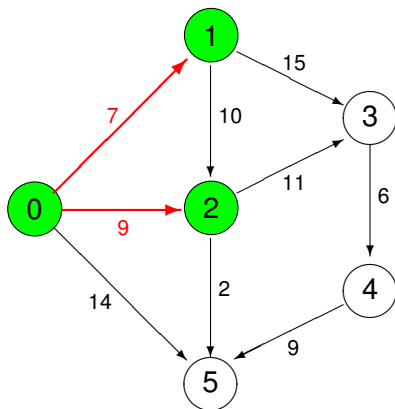


$$d(0) = 0$$

$$d(1) = 7$$

step 2 of Dijkstra's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$,
 $e_{(1,3)} = 15$, $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.



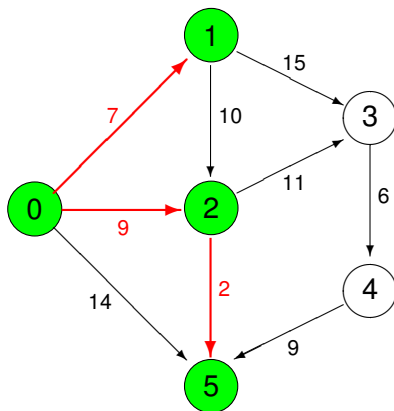
$$d(0) = 0$$

$$d(1) = 7$$

$$d(2) = 9$$

step 3 of Dijkstra's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$,
 $e_{(1,3)} = 15$, $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.



$$d(0) = 0$$

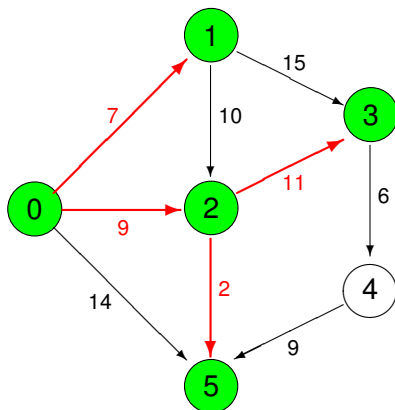
$$d(1) = 7$$

$$d(2) = 9$$

$$d(5) = 11$$

step 4 of Dijkstra's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$,
 $e_{(1,3)} = 15$, $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.



$$d(0) = 0$$

$$d(1) = 7$$

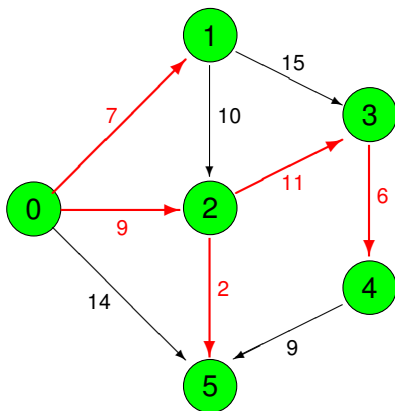
$$d(2) = 9$$

$$d(5) = 11$$

$$d(3) = 20$$

step 5 of Dijkstra's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$,
 $e_{(1,3)} = 15$, $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.



$$d(0) = 0$$

$$d(1) = 7$$

$$d(2) = 9$$

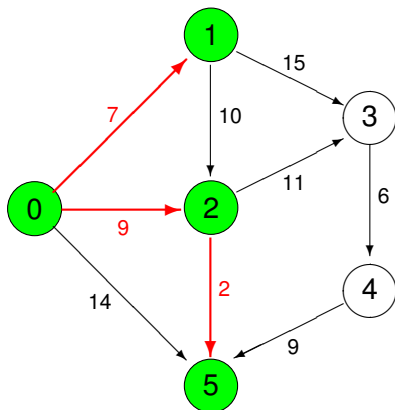
$$d(5) = 11$$

$$d(3) = 20$$

$$d(4) = 26$$

explanation of the correctness, via step 3

In particular, if the set of explored vertices equals three, justify why the path to the fourth vertex in the path from 0 will be the shortest path from 0 to the fourth explored vertex.



$$d(0) = 0$$

$$d(1) = 7$$

$$d(2) = 9$$

$$d(5) = 11$$

After selecting 1 and 2, the next vertex is 5, because 5 is the vertex with shortest length $9 + 2 = 11$, to connect 5 to 0.

8. minimum spanning tree

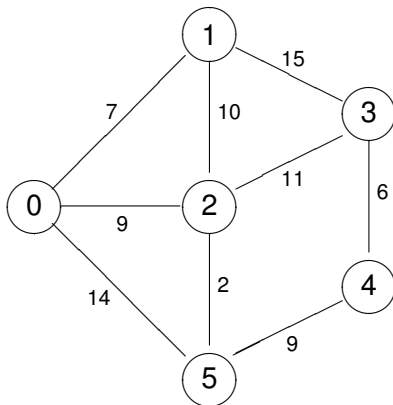
- ① Let G be a weighted graph with 7 vertices 0, 1, 2, 3, 4, 5, 6 with weights w on edges (i, j) written as $e_{(i,j)} = w$ below:
 $e_{(0,1)} = 7$, $e_{(0,2)} = 9$, $e_{(0,5)} = 14$, $e_{(1,2)} = 10$, $e_{(1,3)} = 15$,
 $e_{(2,3)} = 11$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(4,5)} = 9$.

Execute Kruskal's algorithm on the above example.
Draw all steps in the execution of the algorithm.

- ② Illustrate the evolution of the union-find data structure during the execution of Kruskal's algorithm on the above example.
- List the main arguments why using the union-find data structure leads to an $O(m \log(n))$ running time for Kruskal's algorithm, for a graph with n vertices and m edges.

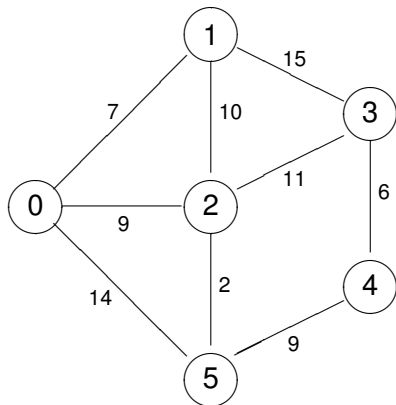
sorting the edges

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



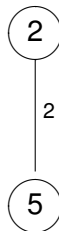
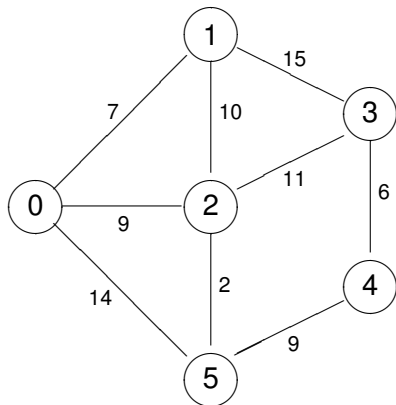
step 1 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



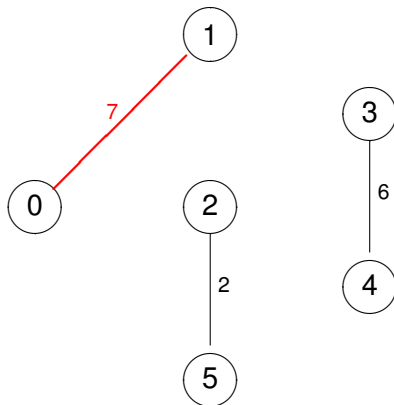
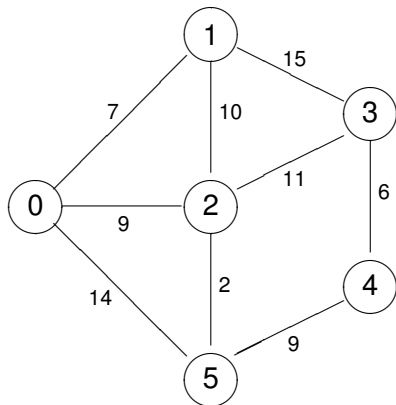
step 2 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



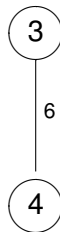
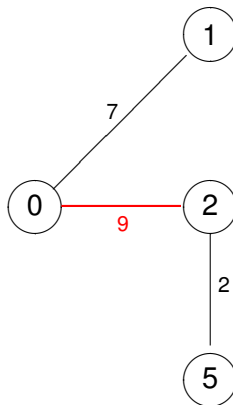
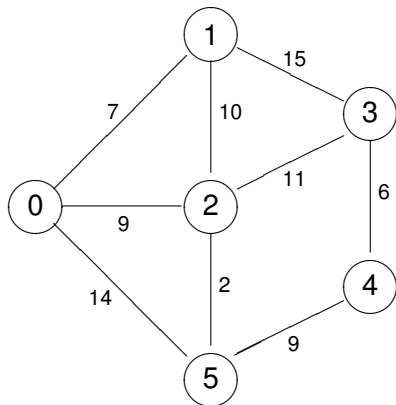
step 3 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



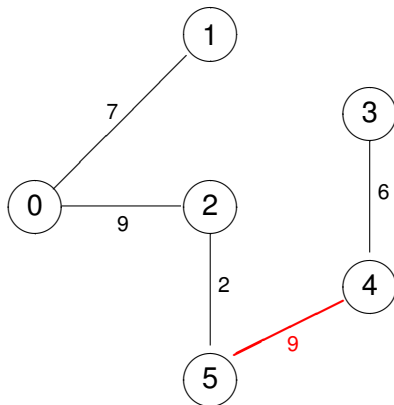
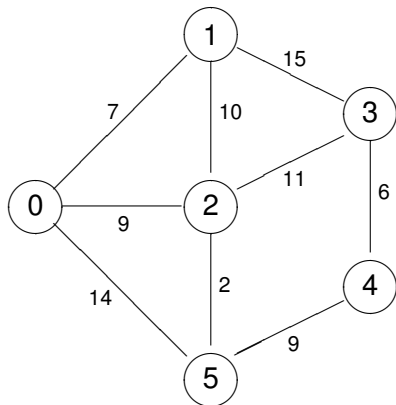
step 4 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



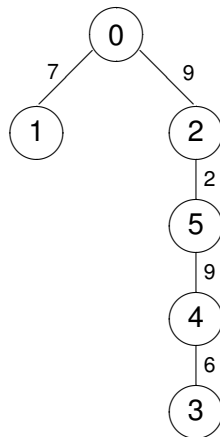
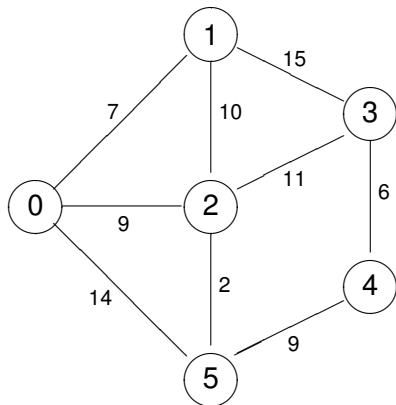
step 5 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



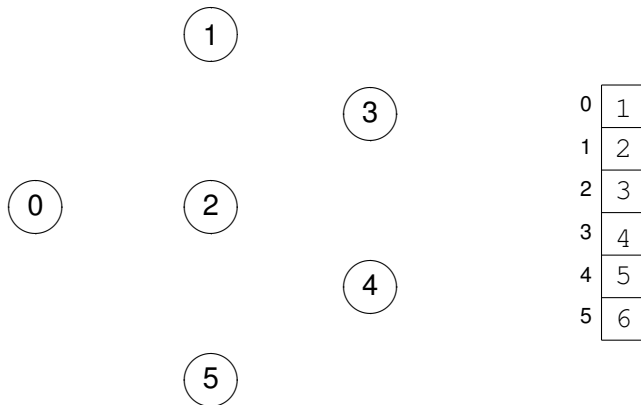
the minimum spanning tree

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



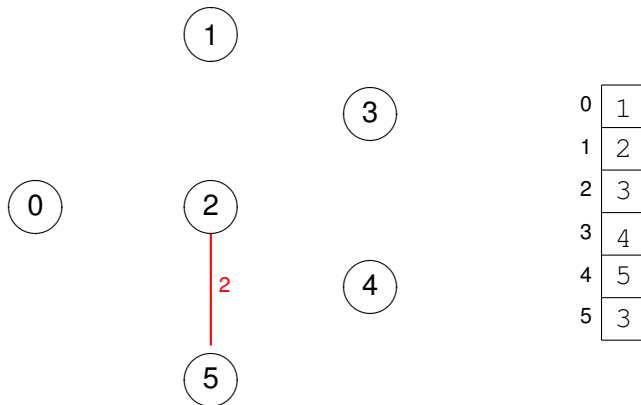
initialization of the union-find data structure

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



step 1 of Kruskal's algorithm

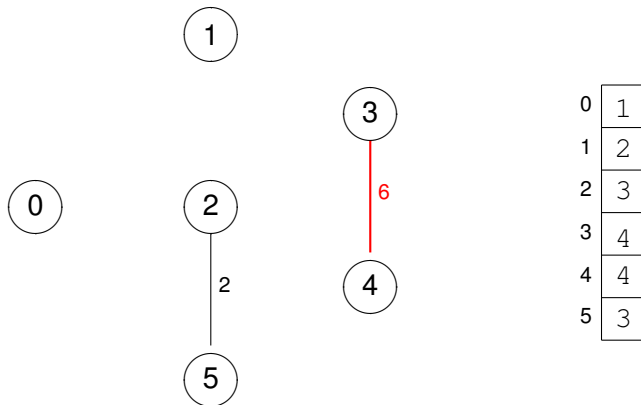
$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



0	1
1	2
2	3
3	4
4	5
5	3

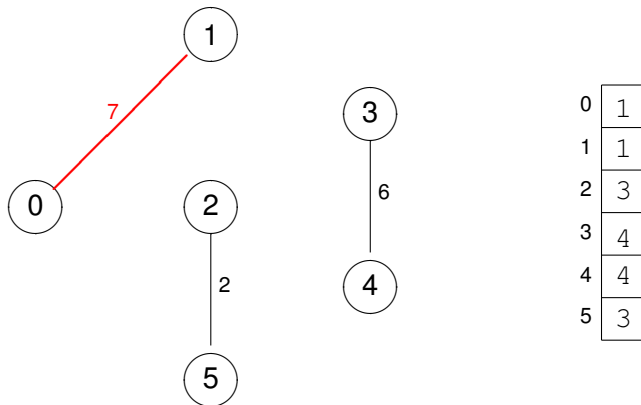
step 2 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



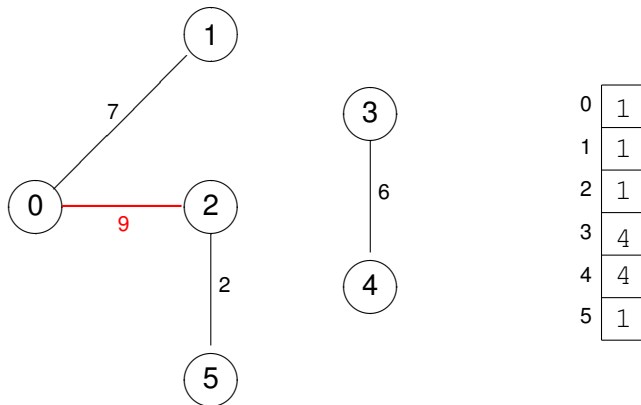
step 3 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



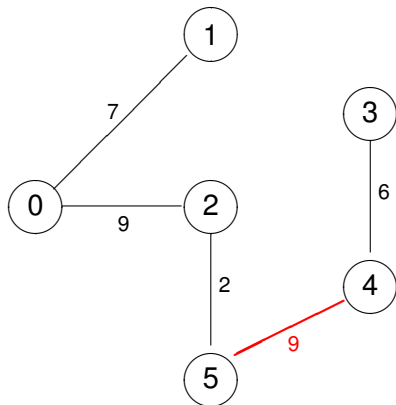
step 4 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



step 5 of Kruskal's algorithm

$V = \{0, 1, 2, 3, 4, 5\}$, $e_{(2,5)} = 2$, $e_{(3,4)} = 6$, $e_{(0,1)} = 7$, $e_{(0,2)} = 9$,
 $e_{(4,5)} = 9$, $e_{(1,2)} = 10$, $e_{(2,3)} = 11$, $e_{(0,5)} = 14$, $e_{(1,3)} = 15$.



0	1
1	1
2	1
3	1
4	1
5	1

on the correctness of Kruskal's algorithm

- 2 List the main arguments why using the union-find data structure leads to an $O(m \log(n))$ running time for Kruskal's algorithm, for a graph with n vertices and m edges.

Answer:

- 1 When taking the union of two disconnected components, the elements in the smaller set take the label of the larger set.
- 2 As a consequence of the above point, any sequence of k union operations takes $O(k \log(k))$ time.
- 3 At most $n - 1$ union operations are performed, which gives $O(n \log(n))$.
- 4 $O(m)$ find operations are performed.
- 5 We have $O(n \log(n))$ is $O(m \log(n))$ and $O(m) + O(m \log(n))$ is $O(m \log(n))$.

9. interval scheduling

Let the tuples $(1, 8)$, $(2, 14)$, $(10, 16)$, $(3, 19)$, $(17, 20)$, and $(18, 21)$ represent the start and finish times of 6 requests for a resource.

- 1 Run the scheduling algorithm to minimize maximum lateness on these requests.
Show all steps in the execution of the algorithm.
- 2 What is the exchange argument used to show optimality?
Illustrate the exchange argument on an example.

running the earliest deadline first algorithm

Let the tuples $(1, 8)$, $(2, 14)$, $(10, 16)$, $(3, 19)$, $(17, 20)$, and $(18, 21)$ represent the start and finish times of 6 requests for a resource.

- 1 Run the scheduling algorithm to minimize maximum lateness on these requests. Show all steps in the execution of the algorithm.

Answer: Reformulate the input, with finish times as deadlines.

Then $(1, 8)$, $(2, 14)$, $(3, 16)$, $(4, 19)$, $(5, 20)$, and $(6, 21)$ are 6 requests given by (index, deadline), and $(7, 12, 6, 16, 3, 3)$ as completion times.

The earliest deadline first algorithm computes start and finish times:

- 1 $s(1) := 0, f(1) := 7, \ell(1) := 0$
- 2 $s(2) := 7, f(2) := 19, \ell(2) := 5$
- 3 $s(3) := 19, f(3) := 25, \ell(3) := 9$
- 4 $s(4) := 25, f(4) := 41, \ell(4) := 22$
- 5 $s(5) := 41, f(5) := 44, \ell(5) := 24$
- 6 $s(6) := 44, f(6) := 47, \ell(6) := 26$

where $\ell(i)$ is the lateness of the i -th request.

the earliest deadline first algorithm is optimal

(1, 8), (2, 14), (3, 16), (4, 19), (5, 20), and (6, 21) are 6 requests given by (index, deadline), and (7, 12, 6, 16, 3, 3) as completion times.

- ② What is the exchange argument used to show optimality?
Illustrate the exchange argument on an example.

Answer: At an inversion, a request with a later deadline is scheduled before a request with an earlier deadline.

The last request has the maximal lateness.

What if we would swap the last two requests?

⑤ $s(6) := 41, f(6) := 44, \ell(6) := 23$

⑥ $s(5) := 44, f(5) := 47, \ell(5) := 27$

As the 6-th request runs 3 units earlier, its lateness dropped by 3.

As the 5-th request runs 3 units later, its lateness increased by 3.

Because of this inversion, the maximum lateness increased by 1.

Review Answers for the Midterm Exam

1 Policies and Overview

- midterm exam policies
- overview of problems, algorithms, data structures
- overview of discrete mathematics

2 Sample Questions with Answers

- on the cost functions of algorithms
- the Gale-Shapley algorithm
- algorithms on graphs
- greedy algorithms
- **divide and conquer**

10. counting inversions

Consider the sequence 8, 5, 2, 4, 6, 1, 3, 7.

- 1 Illustrate the algorithm to count the number of inversions on the above sequence.
- 2 Explain why the counting does not lead to more operations than the number of operations one needs for merge sort.

Illustrate the key argument by referring to the illustration on the above sequence.

counting inversions via divide and conquer

Consider the sequence 8, 5, 2, 4, 6, 1, 3, 7.

- 1 Illustrate the algorithm to count the number of inversions on the above sequence.

Answer: abbreviate `Sort-and-Count` by `S`.

$$\begin{aligned} & S(8, 5, 2, 4, 6, 1, 3, 7) \\ &= S(8, 5, 2, 4) + S(6, 1, 3, 7) \\ &= S(8, 5) + S(2, 4) + S(6, 1) + S(3, 7) \\ &= (1, (5, 8)) + (0, (2, 4)) + (1, (1, 6)) + (0, (3, 7)) \\ &= (1 + 2 + 2, (2, 4, 5, 8)) + (1 + 1, (1, 3, 6, 7)) \\ &= (5, (2, 4, 5, 8)) + (2, (1, 3, 6, 7)) \\ &= (5 + 2 + 4 + 3 + 1 + 1, (1, 2, 3, 4, 5, 6, 7, 8)) \\ &= (16, (1, 2, 3, 4, 5, 6, 7, 8)) \end{aligned}$$

counting inversions via divide and conquer

- 2 Explain why the counting does not lead to more operations than the number of operations one needs for merge sort.

Illustrate the key argument by referring to the illustration on the above sequence.

Answer: When the two sorted sequences are merged, only one extra addition happens each time the item in the second sequence is popped.

The total number of additions is bounded by the number of elements and does not exceed the $O(n)$ cost of the merge step of merge sort.

counting inversions via divide and conquer

- 2 Explain why the counting does not lead to more operations than the number of operations one needs for merge sort.

Illustrate the key argument by referring to the illustration on the above sequence.

Answer: As an illustration, consider the merge step below:

$$\begin{aligned} &= (1, (5, 8)) + (0, (2, 4)) + \dots \\ &= (1 + 2 + 2, (2, 4, 5, 8)) + \dots \end{aligned}$$

When merging (5, 8) with (2, 4), we count each time two inversions, as both 2 and 4 are smaller than the first element of the first sequence.

The $2 + 2$ is added to the one inversion from the (5, 8) sequence.

The number of additions is bounded by the number of elements.

11. divide and conquer

For a number x and positive integer n ,
the direct computation of x^n requires $n - 1$ multiplications.

- 1 Explain how divide and conquer leads to an algorithm which requires $O(\log_2(n))$ multiplications.
- 2 Formulate the algorithm and prove the bound on its running time.

exponentiation via binary expansion

- 1 Explain how divide and conquer leads to an algorithm which requires $\log_2(n)$ multiplications.

Answer: assuming $n = 2^p$, for some nonnegative integer p .

- 1 Obviously, $x^0 = 1$ and $x^1 = x$. For $n > 1$: $x^n = x^{n/2} \times x^{n/2}$.

Under the assumption that $n = 2^p$, repeated squaring suffices, as many times as $p = \log_2(n)$.

- 2 For example, $n = 8$, to compute x^8 , we do

- 1 $z = x \times x$

- 2 $z = z \times z$

- 3 $z = z \times z$

Observe how three multiplications suffice to compute x^8 , and $\log_2(8) = 3$.

exponentiation via binary expansion

- 2 Formulate the algorithm and prove the bound on its running time.

Answer: assuming $n = 2^p$, for some nonnegative integer p .

- 1 The formulation of the algorithm, for $n > 1$ is below:

```
Z := X × X
for k from 2 to  $\log_2(n)$  do
  Z := Z × Z
```

- 2 In the first step of the algorithm, we perform one multiplication.

The loop executes $\log_2(n) - 1$ multiplications.

In total, we have $\log(n)$ multiplications.

The running time for $n = 2^p$ is thus $O(\log_2(n))$.

exponentiation via binary expansion

What if n is not a power of 2? Denote $z(k) = x^k$, then:

$$z(1) = x$$

$$z(2) = x \times x$$

$$z(3) = x^2 \times x = z(2) \times x$$

$$z(4) = x^2 \times x^2 = z(2) \times z(2)$$

$$z(5) = x^4 \times x = z(4) \times x$$

$$z(6) = x^4 \times x^2 = z(4) \times z(2)$$

$$z(7) = x^4 \times x^2 \times x = z(4) \times z(2) \times x$$

No more than $\lceil \log_2(k) \rceil + 1$ multiplications and storage.

$$n = n_p 2^p + n_{p-1} 2^{p-1} + \cdots + n_1 2 + n_0, p = \log_2(n)$$

$$x^n = \prod_{k=1: n_k=1}^p z(k), \text{ at most } p \text{ bits } n_k \text{ in the product}$$

exponentiation via binary expansion

No more than $\lceil \log_2(k) \rceil + 1$ multiplications and storage.

$$n = n_p 2^p + n_{p-1} 2^{p-1} + \cdots + n_1 2 + n_0, p = \log_2(n)$$

$$x^n = \prod_{k=1: n_k=1}^p z(2^k), \text{ at most } p \text{ bits } n_k \text{ in the product}$$

In the $\prod$: $k = 1 : n_k = 1$, take only nonzero bits n_k .

- 1 There are no more than p multiplications for x^n .
- 2 Once we have computed $z(2^k)$ for the highest k , we have already stored all previous $z(2^k)$ values.
- 3 There are p bits in the binary expansion of n , which is the size of z .